

Лабораторна робота №5. Автоматизація процесу в n8n і безпека агента

25 серпня 2026 р.

Мета

Зібрати робочу автоматизацію навчального процесу в самостійно розгорнутому n8n, підключити до неї локальну модель і власні інструменти, а потім атакувати власну систему і показати експериментально, які захисти справді тримають.

Робота виконується **бригадою з трьох осіб**. Лекції 10, 13, 18.

Усе безкоштовне. n8n у режимі self-hosted безкоштовний за ліцензією Sustainable Use для некомерційного й навчального використання; піднімається одним контейнером Docker. Модель — локальна через Ollama, тому жодного платного API і жодної хмарної підписки не потрібно.

Склад бригади і розподіл ролей

Кожен учасник відповідає за свою підсистему, має власний результат і захищає його окремо. Спільним є лише інтеграційний крок і підсумковий звіт.

- **Учасник А — інфраструктура і воркфлоу.** Піднімає n8n у Docker, будує основний воркфлоу з тригером, гілками і обробкою помилок, підключає локальну модель через вузол Ollama.
- **Учасник В — агент та інструменти.** Додає агентний вузол, підключає до нього інструменти й MCP-сервер із лабораторної 4, реалізує межу дозволених дій і підтвердження людини перед незворотним кроком.
- **Учасник С — атаки й захисти.** Складає набір сценаріїв prompt injection із машинно перевірюваними критеріями, міряє частку успішних атак до і після кожного захисту, готує evidence package.

Інтеграційний крок виконують разом: воркфлоу учасника А отримує агентний вузол учасника В, після чого учасник С атакує зібрану систему через дані, які надходять у воркфлоу ззовні. Схему меж довіри бригада малює спільно.

Теоретичні відомості

Воркфлоу і агент — це дві різні відповіді на питання, хто вирішує наступний крок. У воркфлоу маршрут намальований людиною, і модель викликається там, де її поставили; в агенті модель обирає послідовність дій сама. n8n дозволяє мати обидва режими в одній системі, і саме тому на ньому добре видно межу між ними.

- **Автоматизація має власні режими відмови**, незалежні від моделі: тригер спрацював двічі, зовнішній сервіс віддав 500, гілка обробки помилок не написана взагалі. Тому воркфлоу проєктують із явним шляхом помилки, а не лише зі щасливим сценарієм.
- **Недовірений текст стає інструкцією**. У звичайному застосунку потік керування зафіксований у коді, а недовірений ввід є даними для парсера. В агенті потік керування породжується з тексту, і текст цей приходить із документів, з відповідей інструментів і від інших агентів. Пряма інекція живе в запиті користувача, непряма — у даних, що входять у воркфлоу ззовні: у листі, у файлі, у рядку таблиці.
- **Смертельна тріада є булевою умовою:**

$$\text{Risk}_{\text{exfil}} = A \wedge B \wedge C$$

де A — доступ до приватних даних, B — контакт із недовіреним текстом, C — канал назовні. Правило Agents Rule of Two вимагає, щоб сеанс задовольняв **не більше двох** властивостей із трьох. Автоматизація небезпечна саме тим, що зводить усі три разом непомітно: тригер приносить чужий текст, агент має доступ до бази, а вузол відправлення листа стоїть у кінці.

- **Системний промпт не є межею безпеки**. Виміряна ефективність промптових захистів падає під адаптивною атакою: атакувальник ходить другим і бачить ваш захист. Надійний важіль архітектурний — обмежити *наслідок*, а не вхід: заборонити незворотну дію без підтвердження, розділити права, розірвати тріаду.
- **Двадцять сценаріїв — це пілот, а не вимірювання**. Одна атака важить п'ять відсоткових пунктів. Для кожного наслідку критерій успіху має бути машинно перевірюваним — рядок-маркер, факт виклику інструмента, поява захищеного значення у виході, — інакше підрахунок стане предметом суперечки.
- **Захисти міряють поодинці**. Увімкнені послідовно, вони змішують ефект конкретного механізму з порядком додавання, і приписати приріст стає нікому.

Корисні посилання для ознайомлення:

- [n8n](#): розгортання через Docker і ліцензія Sustainable Use;
- Вузол AI Agent та вузол Ollama Chat Model — модель локальна, ключі не потрібні;
- Приклади AI-воркфлоу в документації n8n;
- Greshake K. та ін. [Indirect Prompt Injection](#), AISec 2023;
- OWASP Top 10 для LLM-застосунків і AgentDojo.

Порядок виконання

Позначка [A], [B], [C] вказує відповідального за крок; **[разом]** — крок виконує вся бригада.

1. **[разом] Обрати процес для автоматизації.** Потрібен реальний повторюваний процес навчального відділу: розбір вхідних звернень студентів, підготовка щотижневого дайджесту наказів, перевірка розкладу на конфлікти. Процес описується як тригер, кроки й результат — одним абзацом.
2. **[A] Підняти n8n** у Docker локально, налаштувати збереження даних у томі. Переконаватися, що інтерфейс доступний і воркфлоу зберігаються між перезапусками.
Перевірка: контейнер перезапущено — воркфлоу на місці.
3. **[A] Побудувати базовий воркфлоу:** тригер, щонайменше п'ять вузлів, одна умовна гілка і **явна гілка обробки помилок**. На цьому етапі моделі ще немає: перевіряється сама механіка.
4. **[A] Підключити локальну модель** вузлом Ollama Chat Model і додати крок, де модель класифікує або підсумовує вхідні дані. Зафіксувати, скільки часу займає крок із моделлю проти решти воркфлоу.
5. **[B] Додати агентний вузол** з інструментами: пошук по корпусу лабораторної 2 і МСР-сервер із лабораторної 4. Задати системну інструкцію й перелік дозволених дій.
Перевірка: агент справді викликає інструменти — це видно в журналі виконання n8n, а не лише у фінальній відповіді.
6. **[B] Поставити межу незворотності:** крок, що змінює стан назовні (лист, запис у таблицю, створення звернення), виконується лише після явного підтвердження людини. Заборона реалізується структурою воркфлоу, а не проханням у промпті.
7. **[B] Намалювати межі довіри:** схема, де позначено, звідки в системі заходить недовірений текст, де живуть приватні дані і де є канал назовні. Це і є перевірка тріади для вашої конфігурації.

8. **[C] Скласти сценарії атак:** 20 сценаріїв — 10 прямих і 10 **непрямих**, захованих у даних, які потрапляють у воркфлоу через тригер (текст звернення, вміст файлу, поле таблиці). Три різні наслідки: розкриття системної інструкції, виклик забороненого інструмента, витік даних у вихід. Для кожного — машинно перевірюваний критерій успіху.
9. **[C] Виміряти базову конфігурацію:** частка успішних атак *до* будь-якого захисту, окремо для прямих і непрямих. Без цього числа наступний крок нічого не доводить.
10. **[C] Увімкнути три захисти і виміряти кожен:** розмежування довіреного і недовіреного тексту в промпті; валідація виходу перед незворотним кроком; архітектурне обмеження (розрив тріади — прибрати канал назовні або доступ до приватних даних). Кожен міряється *окремо* відносно бази, потім усі разом.
11. **[разом] Пояснити результат:** які сценарії закрив промпт, які лише архітектура, а які не закрилися взагалі. Для останніх запропонувати залишкові контрзаходи.
12. **[разом] Evidence package:** експорт воркфлоу у JSON, схема меж довіри, таблиця атак до і після кожного захисту, журнал виконання з прикладом спрацювання захисту, висновки до 250 слів із переліком відомих обмежень. У висновку окремо зазначити, що вибірка з 20 сценаріїв є піотною.

Останній крок і є результатом роботи. Список того, що *не* закрилося, цінніший за таблицю з нулями: він показує, що бригада розуміє межі власної системи. Автоматизація, яку ви зібрали, небезпечна саме тим, що працює без людини — і саме це треба довести числами.

Вимоги до задачі та захисту

1. Експорт воркфлоу у JSON плюс README, за яким його можна імпортувати й запустити з нуля. Таблиця «хто що робив» із посиланнями на коміти.
2. Запис демонстрації: успішна атака до захисту і та сама атака після нього.
3. На захисті учасник А показує воркфлоу наживо, В пояснює, чому підтвердження реалізоване структурою, С захищає числа й критерії успіху атак.
4. Бригада разом відповідає, які дві властивості тріади лишилися у фінальній конфігурації і чому саме ці.

Контрольні запитання

1. Чим воркфлоу відрізняється від агента з погляду того, хто обирає наступний крок?
2. Чому в автоматизації непряма інекція небезпечніша за пряму?
3. Які три властивості утворюють смертельну тріаду і яку з них дешевше зняти у вашій системі?
4. Чому системний промпт не є механізмом безпеки?
5. Навіщо міряти кожен захист окремо, а не всі разом?
6. Що таке машинно перевірюваний критерій успіху атаки і чому без нього підрахунок беззмістовний?
7. Тригер спрацював двічі на одному вхідному листі. Що має статися і що станеться без ідемпотентності?
8. Чому 20 сценаріїв не дають підстав стверджувати, що захист «працює на 90 %»?
9. Які сценарії з вашого набору неможливо закрити нічим, крім архітектури?
10. Що входить до evidence package і що саме кожен його елемент доводить?

Література

1. Debenedetti E. та ін. [AgentDojo](#). NeurIPS, 2024.
2. Debenedetti E. та ін. [Defeating Prompt Injections by Design \(CaMeL\)](#), 2025.
3. Nasr M. та ін. [The Attacker Moves Second](#). arXiv:2510.09023, 2025.
4. NIST AI 600-1. [AI RMF: Generative AI Profile](#), 2024.